

Universidad Americana

Facultad de Ingeniería y Arquitectura



Gestión y Administración de Base de Datos

Práctica Integradora de SQL Server

Carlos Alfredo Abea Martínez

Axell Antonio Castillo Tapia

José Gabriel Cano Blandon

Freddy Adrian Peralta Palacios

Marcos Alessandro Lagos Rivera

Profesor: Msc. José Durán

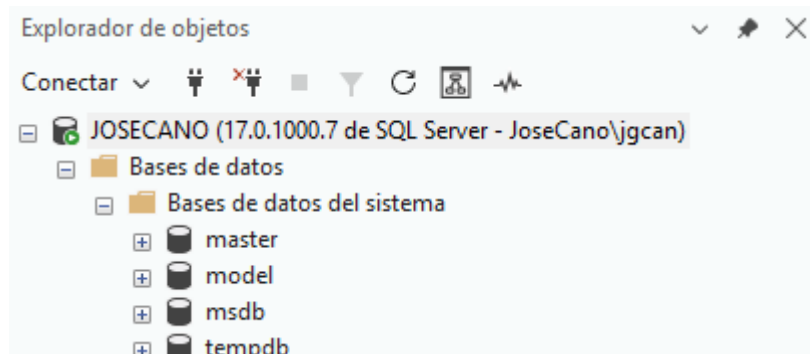
Managua, Nicaragua

24 de Marzo del 2026

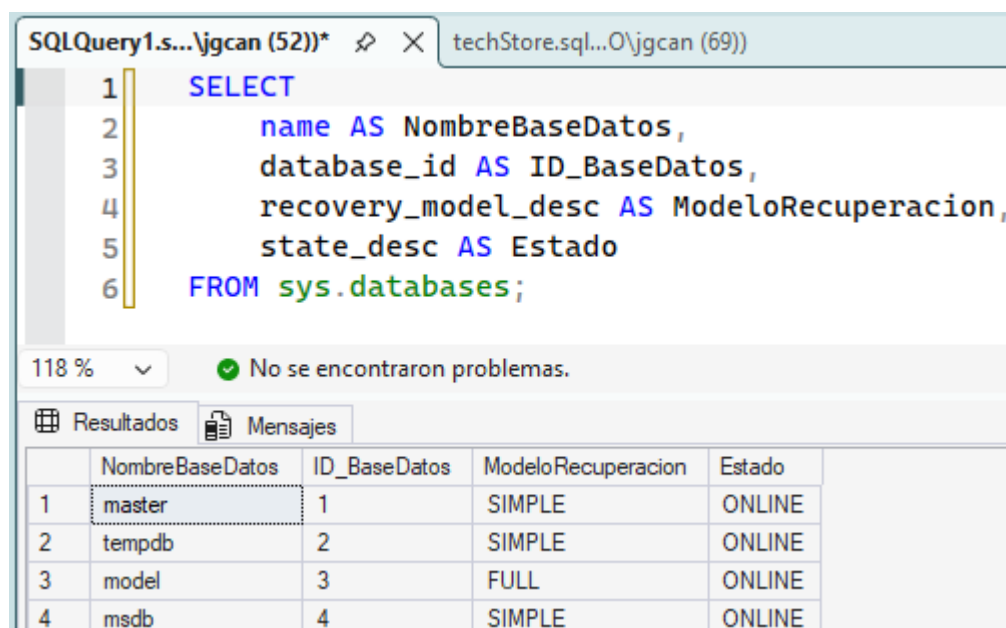
Secuencia de Trabajo por Grupos

1. Construcción de base de datos

Grupo 1



Durante esta fase se realizó una exploración inicial del entorno de SQL Server Management Studio con el objetivo de identificar los componentes principales del servidor y verificar el estado general de las bases de datos.



Se identificaron las bases de datos del sistema: master, model, msdb y tempdb. Estas bases son fundamentales para el funcionamiento interno de SQL Server, ya que almacenan configuraciones, plantillas, información de trabajos, historial y objetos temporales.

SQLQuery1.s... \jgcan (52))* X techStore.sql...O\jgcan (69))

8SELECT

9@@SERVERNAME AS NombreServidor,

10SERVERPROPERTY('Edition') AS EdicionSQLServer,

11SERVERPROPERTY('ProductVersion') AS VersionSQLServer,

12SERVERPROPERTY('ProductLevel') AS NivelProducto,

13SERVERPROPERTY('Collation') AS CollationServidor;

118 % No se encontraron problemas.

Resultados

Mensajes

	NombreServidor	EdicionSQLServer	VersionSQLServer	NivelProducto	CollationServidor
1	JoseCano	Enterprise Developer Edition (64-bit)	17.0.1000.7	RTM	Modern_Spanish_CI_AS

Además, se ejecutó una consulta sobre sys.databases para verificar el nombre de las bases de datos existentes, su modelo de recuperación y su estado actual. También se consultó información general del servidor, como el nombre de la instancia, edición, versión y collation.

SQLQuery1.s... \jgcan (52))* X techStore.sql...O\jgcan (69))

15SELECT

16name AS Login,

17type_desc AS TipoLogin,

18is_disabled AS EstaDeshabilitado,

19create_date AS FechaCreacion

20FROM sys.server_principals

21WHERE type IN ('S', 'U', 'G')

22ORDER BY name;

118 % No se encontraron problemas.

Resultados

Mensajes

	Login	TipoLogin	EstaDeshabilitado	FechaCreacion
1	##MS_PolicyEventProcessingLogin##	SQL_LOGIN	1	2025-10-21 12:53:18.890
2	##MS_PolicyTsqlExecutionLogin##	SQL_LOGIN	1	2025-10-21 12:53:19.013
3	EcoRetos	SQL_LOGIN	0	2026-04-22 22:30:02.240
4	JoseCano\jgcan	WINDOWS_LOGIN	0	2026-04-14 13:50:57.830
5	NT AUTHORITY\SYSTEM	WINDOWS_LOGIN	0	2026-04-14 13:50:57.870
6	NT Service\MSSQLSERVER	WINDOWS_LOGIN	0	2026-04-14 13:50:57.863
7	NT SERVICE\SQLSERVERAGENT	WINDOWS_LOGIN	0	2026-04-14 13:50:58.577
8	NT SERVICE\SQLTELEMETRY	WINDOWS_LOGIN	0	2026-04-14 13:51:00.047
9	NT SERVICE\SQLWriter	WINDOWS_LOGIN	0	2026-04-14 13:50:57.843
10	NT SERVICE\Winmgmt	WINDOWS_LOGIN	0	2026-04-14 13:50:57.850
11	sa	SQL_LOGIN	0	2003-04-08 09:10:35.460

Finalmente, se revisaron los logins registrados en el servidor para obtener una visión inicial de la seguridad y los accesos existentes. Esta información sirve como diagnóstico inicial para continuar con la construcción y administración de la base de datos en las siguientes fases.

Grupo 2:

SQLQuery1.s... \jgcan (52))* techStore.sql... O\jgcan (69))

```
22 ORDER BY name;  
23  
24 SELECT name  
25 FROM sys.databases;
```

80 % No se encontraron problemas.

Resultados Mensajes

	name
1	master
2	tempdb
3	model
4	msdb
5	Ventas_Norte
6	Ventas_Sur
7	Estudiante
8	EmpresaRespaldo
9	EcoRetosDB
10	PruebaBase
11	TechStore

SQLQuery1.s... \jgcan (52))* techStore.sql... O\jgcan (69))

```
24 SELECT name  
25 FROM sys.databases;  
26  
27 SELECT TABLE_NAME  
28 FROM INFORMATION_SCHEMA.TABLES  
29 WHERE TABLE_TYPE = 'BASE TABLE';
```

80 % No se encontraron problemas.

Resultados Mensajes

	TABLE_NAME
1	Cientes
2	Productos
3	Empleados
4	Ventas
5	DetalleVentas
6	TarjetasCientes

En esta fase se llevó a cabo la creación de la base de datos TechStore mediante la ejecución de un script SQL proporcionado. Este script permitió la creación de múltiples tablas relacionadas, incluyendo Clientes, Productos, Ventas, DetalleVentas, Empleados y TarjetasClientes.

SQLQuery1.s... \jgcan (52))* techStore.sql... O\jgcan (69))

```

29 WHERE TABLE_TYPE = 'BASE TABLE';
30
31 SELECT * FROM Clientes;
32 SELECT * FROM Productos;
33 SELECT * FROM Ventas;

```

80 % 3 0 ↑ ↓

Resultados Mensajes

	ClienteID	Nombre	Email	Telefono	Ciudad	FechaRegistro
1	1	Carlos Méndez	carlos@correo.com	8888-1111	Managua	2026-04-28 16:28:54.063
2	2	María López	maria@correo.com	8888-2222	León	2026-04-28 16:28:54.063
3	3	José Ramírez	jose@correo.com	8888-3333	Granada	2026-04-28 16:28:54.063
4	4	Ana Torres	ana@correo.com	8888-4444	Masaya	2026-04-28 16:28:54.063
5	5	Luis Herrera	luis@correo.com	8888-5555	Chinandega	2026-04-28 16:28:54.063

	ProductoID	Nombre	Categoria	Precio	Stock	Activo
1	1	Laptop Lenovo ThinkPad	Computadoras	850.00	15	1
2	2	Mouse Logitech	Accesorios	18.50	80	1
3	3	Teclado Mecánico Redragon	Accesorios	55.00	40	1
4	4	Monitor Dell 24"	Monitores	210.00	20	1
5	5	Disco SSD Kingston 1TB	Almacenamiento	95.00	35	1
6	6	Memoria RAM 16GB	Componentes	70.00	50	1
7	7	Impresora HP LaserJet	Impresoras	180.00	12	1

	VentaID	ClienteID	EmpleadoID	FechaVenta	Total	Estado
1	1	1	1	2026-03-10 00:00:00.000	868.50	Completada
2	2	2	2	2026-03-11 00:00:00.000	265.00	Completada
3	3	3	1	2026-03-12 00:00:00.000	95.00	Completada
4	4	4	3	2026-03-13 00:00:00.000	280.00	Pendiente
5	5	5	2	2026-03-14 00:00:00.000	180.00	Cancelada

Posteriormente, se verificó la correcta creación de la base de datos utilizando consultas sobre sys.databases, así como la existencia de las tablas mediante INFORMATION_SCHEMA.TABLES.

```

35 SELECT
36     c.Nombre AS Cliente,
37     v.FechaVenta,
38     v.Total
39 FROM Ventas v
40 JOIN Clientes c
41     ON v.ClienteID = c.ClienteID;

```

80 % 10 0

Resultados Mensajes

	Cliente	FechaVenta	Total
1	Carlos Méndez	2026-03-10 00:00:00.000	868.50
2	María López	2026-03-11 00:00:00.000	265.00
3	José Ramírez	2026-03-12 00:00:00.000	95.00
4	Ana Torres	2026-03-13 00:00:00.000	280.00
5	Luis Herrera	2026-03-14 00:00:00.000	180.00

Además, se validó la inserción de datos ejecutando consultas SELECT sobre las tablas principales. Finalmente, se realizaron consultas básicas, incluyendo filtros y operaciones JOIN, con el fin de comprobar la integridad y funcionalidad de la base de datos.

Grupo 3: Creación de login y usuario

En esta fase se implementó la seguridad inicial de acceso a SQL Server mediante la creación de un login a nivel de servidor y un usuario dentro de la base de datos TechStore.

```

SQLQuery1.s...\axell (66))* x techStore.sql...O\axell (65))
1 USE master;
2 GO
3
4 CREATE LOGIN usuario_lab
5 WITH PASSWORD = 'LabSQL2026*',
6 CHECK_POLICY = ON;
7 GO

```

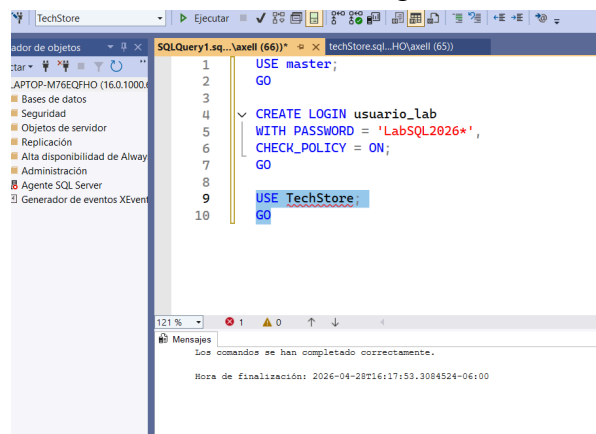
121 % No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-04-28T16:12:01.4102402-06:00

Primero se creó el login usuario_lab desde la base master. Este login permite la autenticación en la instancia de SQL Server. Posteriormente, dentro de la base de datos TechStore, se creó un usuario asociado a dicho login mediante la instrucción CREATE USER FOR LOGIN.



```
1 USE master;
2 GO
3
4 CREATE LOGIN usuario_lab
5 WITH PASSWORD = 'LabSQL2026*',
6 CHECK_POLICY = ON;
7 GO
8
9 USE TechStore;
10 GO
```

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-04-28T16:17:53.3084524-06:00

```
11
12 CREATE USER usuario_lab
13 FOR LOGIN usuario_lab;
14 GO
```

%  No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-04-28T16:19:00.6894641-06:00

```
16 SELECT
17     name AS Usuario,
18     type_desc AS TipoUsuario,
19     create_date AS FechaCreacion
20 FROM sys.database_principals
21 WHERE name = 'usuario_lab';
```

121 %  2  0

Resultados Mensajes

	Usuario	TipoUsuario	FechaCreacion
1	usuario_lab	SQL_USER	2026-04-28 16:19:00.660

Esta separación entre login y usuario permite diferenciar la autenticación del servidor y la autorización dentro de una base de datos específica. El login permite conectarse al servidor, mientras que el usuario permite interactuar con la base de datos TechStore.

```
22 |
23 | USE master;
24 | GO
25 |
26 | SELECT
27 |     name AS Login,
28 |     type_desc AS TipoLogin,
29 |     is_disabled AS EstaDeshabilitado,
30 |     create_date AS FechaCreacion
31 | FROM sys.server_principals
32 | WHERE name = 'usuario_lab';
```

1 % 2 0

Resultados Mensajes

Login	TipoLogin	EstaDeshabilitado	FechaCreacion
usuario_lab	SQL_LOGIN	0	2026-04-28 16:12:01.393

Finalmente, se verificó la existencia del login y del usuario mediante consultas a `sys.server_principals` y `sys.database_principals`. Esta configuración servirá como base para la asignación de roles y permisos en la siguiente fase.

Grupo 4:

Server Name:

Authentication:

User Name:

Password:

☒ Remember Password

Database Name:

Encrypt:

☒ Trust Server Certificate

Color:

```
34 | USE TechStore;
35 | SELECT * FROM Clientes;
```

Inicialmente, se verificó que el usuario `usuario_lab` no contaba con permisos sobre la base de datos, lo cual generó errores al intentar ejecutar consultas.


```
37 | USE TechStore;  
38 | GO  
39 |  
40 | ALTER ROLE db_datareader  
41 | ADD MEMBER usuario_lab;  
42 | GO
```

▼ 1 0 ↑ ↓ ◀

sajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-04-28T16:49:35.7729486-06:00

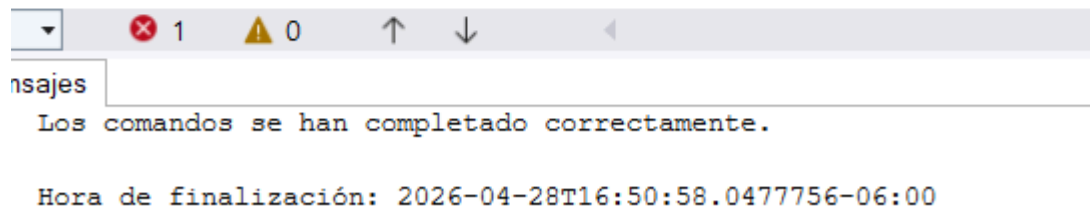
Posteriormente, se asignó el rol db_datareader, permitiendo al usuario realizar consultas SELECT sobre las tablas.

```
· INSERT INTO Clientes (Nombre, Ciudad)  
· VALUES ('Prueba', 'Managua');
```

```
· SELECT * FROM Clientes  
· WHERE Nombre = 'Prueba';
```

Se comprobó que, aunque el usuario podía leer información, no tenía permisos para insertar datos, evidenciado por errores al ejecutar instrucciones INSERT.

```
49  ALTER ROLE db_datawriter
50  ADD MEMBER usuario_lab;
51  GO
```



Para resolver esto, se asignó el rol `db_datawriter`, lo cual permitió realizar operaciones de escritura en la base de datos.

Grupo 5: Auditoría y Gobierno de Datos

En esta fase se realizó una auditoría básica de seguridad sobre la base de datos TechStore, con el objetivo de revisar los accesos existentes, identificar los roles asignados y documentar los permisos configurados.

Primero, se utilizó el procedimiento almacenado del sistema `sp_helpuser` para obtener información general sobre los usuarios existentes en la base de datos.

3	USE TechStore;
4	GO
5	
6	-- Revisar usuarios existentes
7	EXEC sp_helpuser;
8	GO
9	
10	-- Revisar roles asignados a usuarios
11	SELECT
12	DP1.name AS Usuario,
13	DP2.name AS RolAsignado

125 % No se encontraron problemas. Línea: 3

	UserName	RoleName	LoginName	DefDBName	DefSchemaName	UserID	SID
1	analista_ventas	rol_reportes	analista_ventas	master	dbo	6	0x392ECA5CCE6681429C314D857EB939A1
2	analista_ventas	db_owner	analista_ventas	master	dbo	6	0x392ECA5CCE6681429C314D857EB939A1
3	dbo	db_owner	sa	master	dbo	1	0x01
4	guest	public	NULL	NULL	guest	2	0x00
5	INFORMATION_SCHEMA	public	NULL	NULL	NULL	3	NULL
6	invitado_consulta	db_datareader	invitado_consulta	master	dbo	7	0x4C8338FC205C814F91862935E6ADE46C
7	sys	public	NULL	NULL	NULL	4	NULL
8	techstore_app	db_owner	techstore_app	master	dbo	5	0x87017D13BEA1504EAC2FD917B7A6ABFE

Posteriormente, se consultaron las vistas del sistema sys.database_role_members y sys.database_principals para identificar la relación entre usuarios y roles asignados.

11	SELECT
12	DP1.name AS Usuario,
13	DP2.name AS RolAsignado
14	FROM sys.database_role_members DRM
15	JOIN sys.database_principals DP1
16	ON DRM.member_principal_id = DP1.principal_id
17	JOIN sys.database_principals DP2
18	ON DRM.role_principal_id = DP2.principal_id
19	ORDER BY DP1.name;
20	GO

14 % No se encontraron problemas.

	Usuario	RolAsignado
1	analista_ventas	rol_reportes
2	analista_ventas	db_owner
3	dbo	db_owner
4	invitado_consulta	db_datareader
5	techstore_app	db_owner

Se comprobó que el usuario usuario_lab cuenta con los roles db_datareader y db_datawriter, lo cual le permite realizar operaciones de lectura y escritura sobre las tablas de la base de datos. Esta configuración fue aplicada durante la fase anterior para validar el cambio de comportamiento del usuario antes y después de otorgar permisos.

También se revisaron los permisos explícitos registrados en sys.database_permissions y las fechas de creación de usuarios para contar con una trazabilidad básica de los accesos configurados.

23 SELECT
 24 USER_NAME(grantee_principal_id) AS Usuario,
 25 permission_name AS Permiso,
 26 state_desc AS EstadoPermiso,
 27 class_desc AS TipoObjeto,
 28 OBJECT_NAME(major_id) AS Objeto
 29 FROM sys.database_permissions
 30 ORDER BY Usuario, Permiso;
 31 GO
 32
 33 -- Revisar trazabilidad básica de usuarios
 34 SELECT
 35 name AS Usuario,
 36 type_desc AS TipoUsuario,
 37 create_date AS FechaCreacion

94 % No se encontraron problemas.

Resultados Mensajes

	Usuario	Permiso	EstadoPermiso	TipoObjeto	Objeto
1	analista_ventas	CONNECT	GRANT	DATABASE	NULL
2	dbo	CONNECT	GRANT	DATABASE	NULL
3	invitado_consulta	CONNECT	GRANT	DATABASE	NULL
4	public	SELECT	GRANT	OBJECT_OR_COLUMN	sysquery_store_query_hints_2019
5	public	SELECT	GRANT	OBJECT_OR_COLUMN	database_automatic_tuning_configurations
6	public	SELECT	GRANT	OBJECT_OR_COLUMN	query_store_plan_forcing_locations
7	public	SELECT	GRANT	OBJECT_OR_COLUMN	query_store_replicas
8	public	SELECT	GRANT	OBJECT_OR_COLUMN	external_table_partitioning_columns

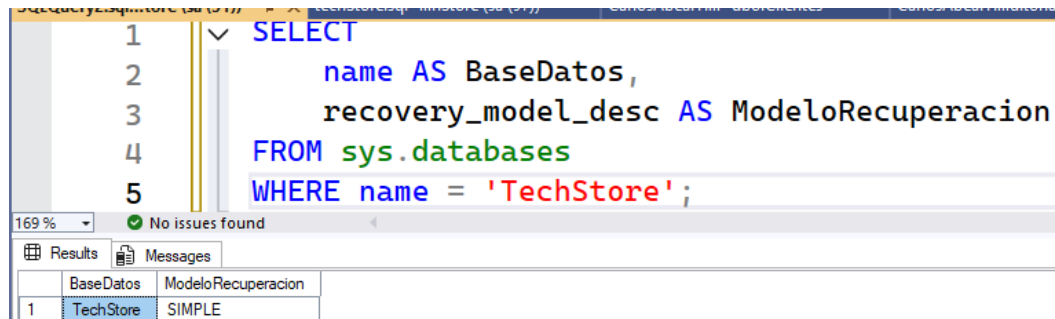
	Usuario	TipoUsuario	FechaCreacion	FechaModificacion
1	dbo	SQL_USER	2003-04-08 09:10:42.287	2026-04-28 16:31:19.177
2	guest	SQL_USER	2003-04-08 09:10:42.317	2003-04-08 09:10:42.317
3	INFORMATION_SCHEMA	SQL_USER	2009-04-13 12:59:11.717	2009-04-13 12:59:11.717
4	sys	SQL_USER	2009-04-13 12:59:11.717	2009-04-13 12:59:11.717

Consulta ejecutada correctamente. DESKTOP-8R9O7SMBDREMOTO

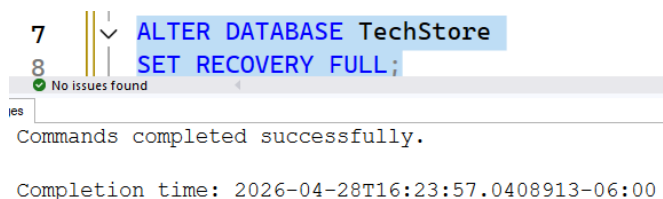
Como resultado, se elaboró una matriz usuario-permisos que permite visualizar de forma clara los accesos existentes y sirve como evidencia para el gobierno de datos dentro de la base TechStore.

Grupo 6. Recuperación, respaldo y retención.

Paso 1. Verificar modelo de recuperación



Como podemos observar el estado de modelo de recuperación esta en simple, sin embargo, necesitamos cambiarlo a full, ya que este guarda toda recuperación total.



En esta fase se evaluó el modelo de recuperación de la base de datos TechStore mediante consultas al catálogo del sistema. Se identificó el modelo de recuperación configurado, el cual determina cómo se registran los cambios y las posibilidades de recuperación ante fallos.

Posteriormente, se realizó un respaldo completo de la base de datos utilizando la instrucción BACKUP DATABASE, generando un archivo físico en el sistema que permite restaurar la información en caso de pérdida o daño.

Back up Full

```
9
10  ✓ BACKUP DATABASE TechStore
11  TO DISK = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\M
12  WITH INIT,
13  NAME = 'Backup Completo TechStore';
```

% No issues found

Messages

Processed 624 pages for database 'TechStore', file 'TechStore' on file 1.
Processed 2 pages for database 'TechStore', file 'TechStore_log' on file 1.
BACKUP DATABASE successfully processed 626 pages in 0.095 seconds (51.439 MB/sec).

Completion time: 2026-04-28T16:30:49.2413657-06:00

Back up Diferencial

```
15  ✓ BACKUP DATABASE TechStore
16  TO DISK = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\Backup\T
17  WITH DIFFERENTIAL, INIT, NAME = 'Backup Diferencial TechStore'
```

No issues found Ln: 17 Ch: 63 Sf

ssages

Processed 112 pages for database 'TechStore', file 'TechStore' on file 1.
Processed 2 pages for database 'TechStore', file 'TechStore_log' on file 1.
BACKUP DATABASE WITH DIFFERENTIAL successfully processed 114 pages in 0.023 seconds (38.552 MB/sec).

Completion time: 2026-04-28T16:39:03.5948632-06:00

Backup de log o transacciones

Adicionalmente, se diseñó una política básica de retención de respaldos, con el objetivo de asegurar la disponibilidad de la información a lo largo del tiempo y permitir su recuperación ante diferentes escenarios.

Grupo 7: Documentación y metadatos.

En esta fase se realizó la documentación completa de la base de datos TechStore, incluyendo sus tablas, campos, tipos de datos, restricciones, llaves primarias, llaves foráneas, índices y procedimientos almacenados.

Para obtener los metadatos se utilizaron herramientas internas de SQL Server, como el procedimiento `sp_help` y las vistas `INFORMATION_SCHEMA.COLUMNS`, `INFORMATION_SCHEMA.TABLES`, `INFORMATION_SCHEMA.TABLE_CONSTRAINTS` y `sys.indexes`. Estas consultas permitieron identificar la estructura lógica y física de los objetos que componen la base de datos.

La base de datos TechStore está compuesta por las tablas Clientes, Productos, Empleados, Ventas, DetalleVentas y TarjetasClientes. Cada tabla cumple una función específica dentro del sistema, permitiendo registrar clientes, productos, empleados, ventas, detalles de venta y métodos de pago asociados a clientes.

También se identificaron relaciones entre tablas mediante claves foráneas, lo cual permite mantener la integridad referencial de la información. Por ejemplo, cada venta está asociada a un cliente y a un empleado, mientras que cada detalle de venta está relacionado con una venta y un producto.

Como resultado de esta fase, se elaboró un diccionario de datos completo que facilita la comprensión, mantenimiento y administración de la base de datos. Esta documentación sirve como insumo técnico para futuras fases de optimización, seguridad, auditoría y escalamiento distribuido.

```

289 USE TechStore;
290 GO
291
292 SELECT
293     TABLE_NAME AS Tabla
294 FROM INFORMATION_SCHEMA.TABLES
295 WHERE TABLE_TYPE = 'BASE TABLE';

```

107 % No se encontraron problemas.

Resultados Mensajes

	Tabla
1	Cientes
2	Productos
3	Empleados
4	Ventas
5	DetalleVentas
6	TarjetasCientes

```

307 SELECT
308     tc.TABLE_NAME AS Tabla,
309     tc.CONSTRAINT_NAME AS Restriccion,
310     tc.CONSTRAINT_TYPE AS TipoRestriccion,
311     kcu.COLUMN_NAME AS Campo
312 FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
313 JOIN INFORMATION_SCHEMA.KEY_COLUMN_USAGE kcu
314     ON tc.CONSTRAINT_NAME = kcu.CONSTRAINT_NAME
315 ORDER BY tc.TABLE_NAME;

```

107 % No se encontraron problemas.

Resultados Mensajes

	Tabla	Restriccion	TipoRestriccion	Campo
1	Cientes	PK__Cientes__71ABD0A77CED9728	PRIMARY KEY	ClientelD
2	DetalleVentas	PK__DetalleV__6E19D6FAE0F6F6BB	PRIMARY KEY	DetalleID
3	DetalleVentas	FK_DetalleVentas_Productos	FOREIGN KEY	ProductID
4	DetalleVentas	FK_DetalleVentas_Ventas	FOREIGN KEY	VentalD
5	Empleados	PK__Empleado__958BE6F081BBC996	PRIMARY KEY	EmpleadoID
6	Productos	PK__Producto__A430AE839D6CC08D	PRIMARY KEY	ProductID
7	TarjetasCientes	PK__Tarjetas__C825069622498FCB	PRIMARY KEY	TarjetaID
8	TarjetasCientes	FK_TarjetasCientes_Cientes	FOREIGN KEY	ClientelD
9	Ventas	FK_Ventas_Cientes	FOREIGN KEY	ClientelD
10	Ventas	FK_Ventas_Empleados	FOREIGN KEY	EmpleadoID
11	Ventas	PK__Ventas__5B41514C72B5150C	PRIMARY KEY	VentalD


```

297 SELECT
298     TABLE_NAME AS Tabla,
299     COLUMN_NAME AS Campo,
300     DATA_TYPE AS TipoDato,
301     CHARACTER_MAXIMUM_LENGTH AS Longitud,
302     IS_NULLABLE AS PermiteNULL,
303     COLUMN_DEFAULT AS ValorPorDefecto
304 FROM INFORMATION_SCHEMA.COLUMNS
305 ORDER BY TABLE_NAME, ORDINAL_POSITION;

```

107 % No se encontraron problemas.

Resultados Mensajes

	Tabla	Campo	TipoDato	Longitud	PermiteNULL	ValorPorDefecto
1	Cientes	CienteID	int	NULL	NO	NULL
2	Cientes	Nombre	nvarchar	100	NO	NULL
3	Cientes	Email	nvarchar	150	NO	NULL
4	Cientes	Telefono	nvarchar	20	YES	NULL
5	Cientes	Ciudad	nvarchar	80	YES	NULL
6	Cientes	FechaRegistro	datetime	NULL	YES	(getdate())
7	DetalleVentas	DetalleID	int	NULL	NO	NULL
8	DetalleVentas	VentaID	int	NULL	NO	NULL
9	DetalleVentas	ProductoID	int	NULL	NO	NULL
10	DetalleVentas	Cantidad	int	NULL	NO	NULL
11	DetalleVentas	PrecioUnitario	decimal	NULL	NO	NULL
12	Empleados	EmpleadoID	int	NULL	NO	NULL
13	Empleados	Nombre	nvarchar	100	NO	NULL
14	Empleados	Cargo	nvarchar	80	NO	NULL
15	Empleados	Email	nvarchar	150	YES	NULL

Consulta ejecutada correctamente.

107 %

No se encontraron problemas.

Resultados

Mensajes

Procedimiento

FechaCreacion

	Tabla	Indice	TipIndice
1	Cientes	PK__Cientes__71ABD0A77CED9728	CLUSTERED
2	DetalleVentas	PK__DetalleV__6E19D6FAE0F6F6BB	CLUSTERED
3	Empleados	PK__Empleado__958BE6F081BBC996	CLUSTERED
4	Productos	PK__Producto__A430AE839D6CC08D	CLUSTERED
5	TarjetasCientes	PK__Tarjetas__825069622498FCB	CLUSTERED
6	Ventas	PK__Ventas__5B41514C72B5150C	CLUSTERED

2. Diccionario de datos

Tabla: Clientes

Campo	Tipo	NuLL	Descripcion
ClienteID	INT IDENTITY	NO	Identificador único del cliente. Llave primaria.
Nombre	NVARCHAR(100)	NO	Nombre completo del cliente.
Email	NVARCHAR(150)	NO	Correo electrónico del cliente.
Telefono	NVARCHAR(20)	SI	Número telefónico del cliente.
Ciudad	NVARCHAR(80)	SI	Ciudad de residencia del cliente.
FechaRegistro	DATETIME	SI	Fecha en que el cliente fue registrado. Valor por defecto: <code>GETDATE()</code> .

Tabla: Productos

Campo	Tipo	NULL	Descripcion
ProductoID	INT IDENTITY	NO	Identificador del producto (PK)
Nombre	NVARCHAR(120)	NO	Nombre del producto
Categoria	NVARCHAR(80)	NO	Categoría
Precio	DECIMAL(10,2)	NO	Precio
Stock	INT	NO	Cantidad disponible
Activo	BIT	SI	Estado del producto

Tabla: Empleados

Campo	Tipo	NULL	Descripcion
EmpleadoID	INT IDENTITY	NO	Identificador del empleado (PK)
Nombre	NVARCHAR(100)	NO	Nombre del empleado
Cargo	NVARCHAR(80)	NO	Puesto
Email	NVARCHAR(150)	SI	Correo
Salario	DECIMAL(10,2)	NO	Salario

Tabla: Ventas

Campo	Tipo	NULL	Descripcion
VentaID	INT IDENTITY	NO	Identificador de la venta (PK)
ClienteID	INT	NO	FK → Clientes
EmpleadoID	INT	NO	FK → Empleados
FechaVenta	DATETIME	SI	Fecha de la venta
Total	DECIMAL(10,2)	NO	Total de la venta
Estado	NVARCHAR(30)	SI	Estado

Tabla: DetalleVentas

Campo	Tipo	NULL	Descripcion
DetalleID	INT IDENTITY	NO	Identificador (PK)
VentaID	INT	NO	FK → Ventas
ProductoID	INT	NO	FK → Productos
Cantidad	INT	NO	Cantidad vendida
PrecioUnitario	DECIMAL(10,2)	NO	Precio en venta

Tabla: TarjetasClientes

Campo	Tipo	NULL	Descripcion
TarjetaID	INT IDENTITY	NO	Identificador (PK)
ClienteID	INT	NO	FK → Clientes
NumeroTarjeta	NVARCHAR(30)	NO	Número de tarjeta
FechaVencimiento	NVARCHAR(10)	NO	Fecha de vencimiento
CVV	NVARCHAR(5)	NO	Código de seguridad

Especificación técnica de TechStore

TechStore es una base de datos transaccional orientada a la gestión de ventas de una tienda tecnológica. Su diseño permite registrar clientes, productos, empleados, ventas y detalles de cada transacción.

La tabla Clientes almacena la información personal y de contacto de los compradores. La tabla Productos administra el catálogo de artículos disponibles, incluyendo categoría, precio, stock y estado activo. La tabla Empleados registra al personal encargado de realizar ventas.

La tabla Ventas funciona como encabezado de cada transacción, relacionando al cliente y al empleado. La tabla DetalleVentas almacena los productos vendidos en cada venta, permitiendo representar una relación entre ventas y productos. Finalmente, la tabla TarjetasClientes almacena información de tarjetas asociadas a clientes.

El modelo utiliza llaves primarias para identificar registros únicos y llaves foráneas para asegurar la relación entre entidades. Además, cuenta con un índice sobre el campo Activo de Productos y procedimientos almacenados para búsqueda y reportes.

Grupo 8.

En esta fase se organizó el desarrollo de la base de datos TechStore mediante una simulación de control de versiones. Esto significa que cada parte del trabajo se separó por etapas, permitiendo identificar qué cambio se hizo, por qué se hizo y qué impacto tuvo dentro del proyecto. Esta práctica es importante porque evita desorden en los scripts y facilita corregir errores si algo falla.

El control de versiones permite llevar un historial claro de la evolución de la base de datos. En lugar de tener todo el código SQL mezclado en un solo archivo, se divide en versiones como v1, v2, v3 y así sucesivamente. Cada versión representa una fase específica del proyecto, por ejemplo, la creación de la base de datos, la inserción de datos, la configuración de seguridad, la asignación de permisos, el respaldo, la documentación y la optimización.

La aplicación de esta estrategia permite simular un entorno profesional de trabajo, similar al que se utiliza en proyectos reales de desarrollo y administración de bases de datos. Además, facilita que otros integrantes del equipo puedan comprender el orden en que fue construida la base de datos y puedan reutilizar los scripts correctamente.

Versión	Cambio realizado	Motivo del cambio	Impacto
v1	Creación de la base de datos y tablas	Construir la estructura inicial	Base de datos funcional
v2	Inserción de registros	Validar el funcionamiento de las tablas	Datos disponibles para pruebas
v3	Creación de login y usuario	Implementar seguridad de acceso	Usuario asociado a la base
v4	Asignación de roles y permisos	Controlar lectura y escritura	Acceso regulado
v5	Revisión de auditoría	Documentar accesos existentes	Mayor trazabilidad
v6	Backup y recuperación	Proteger la información	Respaldo disponible
v7	Documentación y metadatos	Comprender la estructura de la base	Diccionario de datos
v8	Optimización	Mejorar rendimiento e integridad	Base más eficiente
v9	Arquitectura distribuida	Proponer escalamiento por nodos	Modelo conceptual ampliado

Grupo 9. Optimización de la base de datos

En esta fase se realizó un análisis general de la base de datos **TechStore** para identificar mejoras de rendimiento, integridad y seguridad. La optimización permite que la base sea más rápida, confiable y segura.

```
USE TechStore;
```

```
GO
```

Primero, se recomienda crear índices en campos usados frecuentemente en búsquedas, como el nombre de clientes y productos. Esto ayuda a que las consultas sean más rápidas cuando la base crezca.

```
CREATE INDEX IX_Clientes_Nombre
```

```
ON Clientes(Nombre);
```

```
GO
```

```
CREATE INDEX IX_Productos_Nombre
```

```
ON Productos(Nombre);
```

```
GO
```

También se recomienda crear índices en las llaves foráneas, ya que estas columnas se usan mucho en relaciones entre tablas, especialmente en consultas con **JOIN**.

```
CREATE INDEX IX_Ventas_ClienteID
```

```
ON Ventas(ClienteID);
```

```
GO
```

```
CREATE INDEX IX_Ventas_EmpleadoID
```

```
ON Ventas(EmpleadoID);
```

```
GO
```

```
CREATE INDEX IX_DetalleVentas_VentaID
```

```
ON DetalleVentas(VentaID);
```

```
GO
```

```
CREATE INDEX IX_DetalleVentas_ProductoID
```

```
ON DetalleVentas(ProductoID);
```

```
GO
```

En segundo lugar, se recomienda agregar restricciones **CHECK** para evitar datos incorrectos. Por ejemplo, un producto no debería tener precio negativo ni stock menor que cero.

```
ALTER TABLE Productos
```

```
ADD CONSTRAINT CHK_Productos_Precio
```

```
CHECK (Precio > 0);
```

```
GO
```

```
ALTER TABLE Productos
ADD CONSTRAINT CHK_Productos_Stock
CHECK (Stock >= 0);
GO
```

También se recomienda validar los datos de ventas. El total de una venta debe ser mayor o igual a cero, y la cantidad vendida debe ser mayor que cero.

```
ALTER TABLE Ventas
ADD CONSTRAINT CHK_Ventas_Total
CHECK (Total >= 0);
GO
```

```
ALTER TABLE DetalleVentas
ADD CONSTRAINT CHK_DetalleVentas_Cantidad
CHECK (Cantidad > 0);
GO
```

```
ALTER TABLE DetalleVentas
ADD CONSTRAINT CHK_DetalleVentas_PrecioUnitario
CHECK (PrecioUnitario > 0);
GO
```

Finalmente, se identificó un riesgo de seguridad en la tabla **TarjetasClientes**, ya que almacena datos sensibles como número de tarjeta y CVV. Como mejora, se recomienda no almacenar el CVV.

```
ALTER TABLE TarjetasClientes
DROP COLUMN CVV;
GO
```

En esta fase se propone una arquitectura distribuida para la base de datos TechStore, con el objetivo de permitir que la empresa pueda operar en diferentes sucursales conectadas entre sí. Una base de datos distribuida permite que la información se almacene o procese en más de un nodo, facilitando el trabajo de distintas ubicaciones sin depender completamente de un único punto físico.

En este modelo, cada sucursal funciona como un **nodo**. La sucursal Managua y la sucursal León pueden registrar clientes, productos y ventas de forma local. Luego, esos datos pueden sincronizarse con un servidor central, donde se consolida la información general de la empresa.

Se propone que cada sucursal almacene localmente sus propias ventas y detalles de ventas, ya que estos datos se generan directamente en cada ubicación. Por ejemplo, las ventas realizadas en Managua se guardarían primero en el nodo Managua, mientras que las ventas realizadas en León se guardarían en el nodo León.

Sucursal Managua:

- Clientes registrados en Managua
- Ventas realizadas en Managua
- Detalles de ventas de Managua

Sucursal León:

- Clientes registrados en León
- Ventas realizadas en León
- Detalles de ventas de León

Servidor Central:

- Consolidación general de clientes
- Reportes globales de ventas
- Inventario general
- Administración y respaldo

Por otro lado, el servidor central puede encargarse de consolidar la información de todas las sucursales. Esto permitiría generar reportes generales, revisar ventas totales, controlar inventarios y mantener respaldos de la información.

La comunicación entre nodos se realizaría mediante un proceso de sincronización periódica. Esto significa que cada cierto tiempo las sucursales enviarían sus datos nuevos o modificados hacia el servidor central. De la misma manera, el servidor central podría devolver información actualizada sobre productos, precios o inventario.

Sucursal Managua → envía ventas → Servidor Central

Sucursal León → envía ventas → Servidor Central

Servidor Central → envía actualizaciones → Sucursales

Esta sincronización puede realizarse de forma programada, por ejemplo, cada hora o al finalizar el día, dependiendo de las necesidades de la empresa.

Uno de los principales desafíos de una arquitectura distribuida es mantener la **consistencia de los datos**. Esto significa asegurar que la información sea correcta y esté actualizada en todos los nodos.

Por ejemplo, si dos sucursales modifican el stock de un mismo producto al mismo tiempo, puede generarse un conflicto. Para evitarlo, se pueden definir reglas de sincronización, como priorizar los cambios del servidor central o registrar cada movimiento de inventario con fecha, hora y sucursal.

La base de datos **TechStore** ya cuenta con tablas que pueden adaptarse a una arquitectura distribuida, como **Clientes**, **Productos**, **Ventas**, **DetalleVentas** y **Empleados**. Para mejorar la distribución, se recomienda agregar un campo llamado **SucursalID** en las tablas principales, especialmente en **Ventas**, **Clientes** y **Empleados**.

Este campo permitiría identificar en qué sucursal se originó cada registro, facilitando la sincronización, la auditoría y la generación de reportes por ubicación.

Anexos

Link a git hub:

[Abbea2007/BaseDeDatosPracticaIntegradora](https://github.com/Abbea2007/BaseDeDatosPracticaIntegradora)